Capstone Project Proposal: *Après* - A Hospitality Staff Management Platform

Student: Johnny Clifford
Course: Software Development
Date: 10/19/2026

Project Idea
Having spent years working in hospitality, I know the rhythm of a shift well: the rush, the handoff, the after-work debrief. I also know its frustrations. Schedules poorly formatted. Shift swaps facilitated in a largely dormant group chat. Tips divided on a seemingly random basis.
Après is a full-stack web application that brings shift scheduling, team communication, and tip distribution into one place. It's designed around the three roles that define hospitality work: *Manager, Staff,* and *Owner.* Where enterprise tools like 7shifts and HotSchedules are expensive and over-engineered for small teams, *Après* is built for the 10–50 person venue that just needs something that works.
The name comes from *après-ski*—the tradition of gathering after the work is done. That's the spirit of the app: a shared space that keeps a team connected before, during, and after every shift.

Target Audience
Independent hotels, restaurants, and resorts with 10–50 employees currently relying on manual scheduling. Within these venues, the app serves three distinct roles:
- *Managers* — need to build schedules quickly, approve swaps, communicate with staff, and calculate tips
- *Staff* — need clear visibility of their shifts, an easy way to request swaps, and transparent tip earnings
- *Owners* — need visibility into labor costs, coverage gaps, and tip distribution

Core Features
- *Shift Scheduling (Manager)* — Drag-and-drop weekly calendar with shift creation, editing, and assignment by role and availability. Automatic conflict detection for double-booking, unavailability, and overtime risk.
- *Availability & Shift Swaps (Staff)* — Weekly availability submission and coworker swap requests, with a manager approval workflow and status notifications.
- *Real-Time Team Communication* — Shift-specific chat threads and manager announcements, powered by WebSockets for live delivery, typing indicators, and read receipts.
- *Tip Pooling Calculator* — Configurable rules based on hours worked, role weighting, or a hybrid model, with a transparent per-staff breakdown.
- *Task Checklists* — Role-specific opening/closing checklists and assignable side-work with per-shift completion tracking.
- *Owner Dashboard* — Weekly labor cost vs. scheduled hours, coverage gap alerts, overtime warnings, and tip summaries with visual charts.
- *Authentication & Role-Based Access* — JWT authentication with three permission tiers (Admin, Manager, Staff). Managers scope to their department; owners see all locations.

Technology Stack

Frontend

- *React* — Component-based UI for multi-role dashboards
- *React Router* — Protected routes per role
- *Redux Toolkit* — Centralised state for auth, shifts, and live socket events
- *TailwindCSS* — Rapid, responsive styling
- *FullCalendar* — Drag-and-drop scheduling interface
- *Recharts* — Owner dashboard visualisations
- Socket.IO Client — Persistent WebSocket connection

Backend

- *Node.js* + *Express* — Non-blocking I/O suited to real-time, multi-user apps
- *Socket.IO* — WebSocket server for chat, notifications, and live shift updates
- *JWT* + *bcrypt* — Stateless authentication with secure password hashing
- *Jest* + *Supertest* — Unit and integration testing

Database

MongoDB with *Mongoose* — Flexible schema for evolving shift, availability, and chat data

Hosting (Zero-Cost Stack)

Vercel (frontend), Render (backend), MongoDB Atlas (database, M0 free tier), UptimeRobot (free monitoring) — Pings the backend every 5 minutes to prevent Render cold starts and provide external health monitoring

Key infrastructure decision

The backend on Render connects to MongoDB Atlas rather than a Render-hosted database, avoiding Render's 30-day free database expiry and ensuring data persists for the life of the project. To mitigate Render's sleep-on-idle behaviour, UptimeRobot keeps the service warm and doubles as a lightweight monitoring layer.

Security & Scalability Notes

Authentication uses JWT with bcrypt-hashed passwords, all secrets are stored in environment variables, CORS is configured per environment, and auth routes are rate-limited. On scalability, the current setup is intentionally scoped to a portfolio demo on MongoDB Atlas M0. A production path would move to an M10+ cluster with connection pooling and a paid Render tier to eliminate cold starts — I plan to document in the README as a known trade-off.

Testing Strategy

Testing focuses on business logic first, since that's where the riskiest code lives:

- *Unit tests (Jest):* Tip pooling calculation, shift conflict detection, availability validation, RBAC middleware
- *Integration tests (Supertest):* Auth routes, shift CRUD, swap approval workflow
- *Component tests (React Testing Library):* Shift cards, availability forms, role-gated UI
- *Manual QA:* Cross-browser (Chrome, Firefox, Safari) and mobile responsiveness

Target: ≥70% coverage on backend business logic, with a documented debugging log in the devlog.

Summary

Après addresses a real operational problem in an industry I know firsthand. It goes well beyond a typical CRUD app by requiring role-based access control, real-time communication, scheduling logic, and financial calculations — all built with the MERN stack, industry-standard practices, and a zero-cost deployment stack with monitoring and security considerations documented.